# OndaPlay: notas de la presentación

Grupo GLMR

GA-TL-02 — Aplicaciones Distribuidas (ISR-701), 11 de noviembre de 2026

Repositorio: `https://github.com/yaser-uwu/GLMR`

## Escenario

OndaPlay es una plataforma de video por suscripción. El diagrama del grupo muestra cómo se comunican sus servicios y está dividido en tres partes: (1) acceso, licencias y suscripciones, (2) telemetría y recomendaciones y (3) entrega de video. Cada integrante explica su parte abajo.

## Notas de presentación García: acceso, catálogo y licencias

**Qué hice.** Diseñé el flujo que sigue el Reproductor para iniciar sesión, consultar el catálogo y obtener una licencia para el título elegido. La Entrada web, implementada en Spring Boot, comunica al Reproductor con Catálogo y con las réplicas de Autenticación y licencias. Este último servicio consulta a Facturación cuando necesita confirmar la vigencia de la suscripción. Con la licencia firmada, el Reproductor pide el manifiesto y los fragmentos HLS a la CDN por HTTPS sobre HTTP/2.

**Cómo funciona.**

1. **Inicio de sesión (K1–K2).** El Reproductor envía el usuario y la contraseña por HTTPS a la Entrada web, que ejecuta `IniciarSesion` por gRPC. El Reproductor recibe una sesión válida o un rechazo de credenciales.
2. **Consulta del catálogo (K3–K4).** El Reproductor pide el catálogo por HTTPS con filtros y paginación. La Entrada web llama a `ListarTitulos` por gRPC y devuelve los identificadores, títulos y metadatos para mostrarlos en pantalla.
3. **Solicitud de licencia (K5–K6).** El Reproductor envía la sesión y el identificador del contenido por HTTPS. La Entrada web llama a `EmitirLicencia` por gRPC. Si se confirma que la suscripción está activa, Autenticación y licencias firma una licencia que identifica al usuario, al contenido y su fecha de expiración.
4. **Verificación de la suscripción (K7).** Si el servicio necesita confirmar la vigencia, llama a `VerificarSuscripcion` por gRPC con el identificador del usuario. Facturación devuelve el estado, su versión, el fin de la suscripción y `validUntil`, que indica hasta cuándo se puede confiar en ese estado. Si la suscripción está inactiva, se rechaza el acceso. Si no se puede confirmar el estado cuando hace falta, se devuelve un error temporal y el Reproductor puede reintentar.

**Cómo se plantean R2 y R4.** Para R2, el presupuesto de arranque propuesto con la entrega de video reserva 450 ms desde la solicitud de licencia hasta su respuesta. Esa cifra es una meta que se debe medir bajo carga, incluida la consulta a Facturación cuando corresponda. Para R4, Facturación publica avisos versionados por Kafka de forma asíncrona. Al recibir una versión nueva, Licencias

invalida el estado anterior de la suscripción. La licencia vence como máximo a los 4 minutos y queda limitada también por `validUntil` y por el fin de la suscripción.

**Por qué así.** Usé REST sobre HTTPS entre el Reproductor y la Entrada web, y llamadas gRPC sobre HTTP/2 con TLS entre servicios. Las solicitudes son síncronas porque el cliente necesita la sesión para continuar, los títulos para mostrar el catálogo y la licencia antes de pedir el video.

**Qué se sacrifica.** Cuando hay que consultar a Facturación, esa llamada agrega tiempo al arranque. Si la consulta falla y el servicio carece de un estado vigente, incluso un usuario con suscripción activa tiene que reintentar.

## Notas de presentación Lombeida: entrega de video

**Qué hice.** Dibujé el recorrido desde que el Reproductor recibe la licencia de la Entrada web hasta que muestra el primer cuadro. Incluí la Entrada CDN, dos nodos de borde como ejemplo de una red con más nodos y el Origen de video. Mi parte responde a los 50 000 espectadores simultáneos (R1), al arranque en menos de 2 segundos incluida la licencia (R2) y a la caída de un nodo de borde (F3).

**Cómo funciona.**

1. La Entrada web devuelve al Reproductor una licencia firmada por HTTPS. Es la respuesta síncrona del flujo de Kevin: el Reproductor la espera antes de pedir el video.
2. El Reproductor pide a la Entrada CDN el manifiesto HLS y después el primer fragmento por HTTPS. Espera esas respuestas porque necesita los datos para comenzar a reproducir.
3. La Entrada CDN elige un nodo de borde sano. El nodo A responde desde su caché NGINX; si no tiene el manifiesto o el fragmento, lo obtiene del Origen por HTTPS y lo guarda para otras solicitudes.
4. El Reproductor almacena temporalmente los datos, decodifica el fragmento y muestra el primer cuadro. Este ùltimo paso ocurre dentro del cliente y no es una comunicación por red.

**Cómo se plantean R1 y R2.** Los nodos A y B representan una red de varios nodos, no el total de servidores. La caché reparte la entrega y evita que los 50 000 espectadores pidan cada fragmento directamente al Origen. Como ejemplo de dimensionamiento, a 3 Mb/s por espectador se requieren unos 150 Gb/s de salida agregada; la capacidad real debe medirse por región y conservar margen si un nodo falla. Para R2 propongo un presupuesto de 450 ms para licencia con Kevin, 150 ms para ruta y manifiesto, 650 ms para el primer fragmento, 250 ms para búfer y decodificación y 300 ms de margen: 1 800 ms en total. Son metas de diseño que deben acordarse con Kevin y comprobarse bajo carga.

**Qué pasa en F3.** Si el nodo A cae, la Entrada CDN deja de seleccionarlo y el Reproductor reintenta la solicitud pendiente por la misma dirección de la CDN. Otro nodo sano, representado por B, responde desde su caché o consulta al Origen. Se conserva la posición de reproducción, aunque la detección y el reintento pueden causar una pausa.

**Por qué así.** HLS divide el video en un manifiesto y fragmentos que pueden entregarse por HTTPS y guardarse en los bordes de la CDN. Esto permite acercar el contenido a los espectadores y repartir la carga entre nodos. El balanceador evita seguir enviando solicitudes a un nodo que ha fallado.

**Qué se sacrifica.** Un fallo de caché obliga a consultar al Origen y puede aumentar el tiempo de arranque. La ruta alternativa también requiere tiempo de detección y capacidad libre. Por eso el objetivo de R2 no se puede prometer para todos los fallos sin pruebas de carga y de recuperación.

## Notas de presentación Manotoa: telemetría y recomendaciones

**Qué hice.** Diseñé la arquitectura del módulo de telemetría y recomendaciones utilizando WebSockets para ingesta, un Message Broker (Kafka/Redis) como búfer y procesadores workers con base de datos analítica.

**Cómo funciona.**

1. El reproductor envía eventos por WebSockets hacia la capa de ingesta de forma asíncrona.
2. El Message Broker almacena temporalmente los datos para garantizar la tolerancia a fallos.
3. Los procesadores workers distribuyen y transforman la información para alimentar una base de datos analítica.
4. El servicio de recomendaciones consume los datos procesados para actualizar el catálogo y los perfiles de usuario.

**Cómo se cumple con R3, F2, y R5.**

- **R3:** Maneja 5 000 eventos/s con menos de 1 % de pérdida mediante canales persistentes.
- **F2:** Retiene los datos en el búfer del broker durante las 2 horas de caída del servicio de recomendaciones.
- **R5:** Procesa por lotes los eventos para actualizar sugerencias en hasta 15 minutos.

**Por qué así.** Separa los procesos rápidos del procesamiento pesado, desacoplando los componentes para evitar bloqueos y garantizar alta disponibilidad ante fallos prolongados.

**Qué se sacrificó.** Se asumió eventual consistencia y un ligero retraso acumulado en el tiempo de procesamiento de las recomendaciones tras la recuperación de fallas masivas.

## Notas de presentación Reinoso: renovación, pagos y cancelación

**Qué hice.** Dibujé cómo se renueva una suscripción y cómo se avisa al servicio de licencias para que una suscripción vencida o cancelada no pueda iniciar reproducciones nuevas en menos de 5 minutos (requisito R4).

**Cómo funciona.**

1. El usuario renueva desde el Reproductor. La Entrada web pasa el pedido a Facturación por gRPC y espera la respuesta.
2. Facturación cobra a la Pasarela de pagos por HTTPS y espera la respuesta. El cobro lleva una clave que evita cobrar dos veces si el pedido se repite.
3. Facturación avisa el cambio por Kafka, sin esperar respuesta. Licencias lee el aviso y actualiza su copia del estado.
4. Si Licencias duda de su copia, pregunta a Facturación por gRPC.

**Cómo se cumple R4.** Cada licencia dura como máximo 4 minutos y nunca más allá del vencimiento de la suscripción. Si Licencias pierde la conexión con Kafka o lee los avisos con más de 30 segundos de atraso, pregunta a Facturación antes de emitir una licencia. Si no puede confirmar el estado, no emite una licencia nueva y el usuario reintenta. En el peor caso, el bloqueo ocurre a los 4 minutos y 30 segundos, bajo el límite de 5.

**Por qué así.** Consultar a Facturación en cada reproducción también cumpliría R4, pero el arranque dependería de Facturación: si se cae o se demora, nadie podría empezar a ver nada (requisitos R2 y R6). Con el aviso por Kafka, Licencias responde con su copia.

**Qué se sacrifica.** Si Facturación está caída y Licencias no puede confirmar el estado, un usuario con suscripción válida tiene que reintentar.

## Declaración de uso de inteligencia artificial

Durante la elaboración de este informe se utilizó ChatGPT (OpenAI) como apoyo para organizar y redactar parte de las notas de presentación, y para revisar su concordancia con el diagrama grupal de OndaPlay.